



ACCURACY

V 3.0.0

DEPLOYMENT PRESENTATION

PIPELINE SCORING · STAGE A + STAGE B

SOMMAIRE

Table des matières

01	TL;DR	p. 03
02	Pourquoi cette release ?	p. 05
03	Comment ça marche, pédagogiquement	p. 07
04	Ce qu'on déploie concrètement	p. 11
05	Rollout en 3 phases	p. 13
06	Métriques de succès chiffrées	p. 14
07	Limitations honnêtes	p. 17
08	Annexes	p. 18

AUDIENCE Équipe ingénierie commodities-compass.

OBJET Présenter le déploiement du nouvel algo de scoring `accuracy v3.0.0` avant la phase d'implémentation.

COMPANION DOCS La spec complète et le journal de R&D sont dans `COMPASS_V9_PROD_INTEGRATION.md` .

STATUT Proposition — à valider avant `git checkout -b feat/accuracy-v3` .

01 · SYNTHÈSE

TL;DR

On remplace l'algo de décision actuel (`legacy v1.0.1` , **31% de précision en avril 2026**) par un pipeline en deux étages. La logique : séparer la **production du score** (interprétable, déterministe, auditable) de la **validation contextuelle** (non-linéaire, apprise sur historique). Chaque étage fait une chose, bien.

1. **Stage A — le scoreur.** Un polynôme à 16 features (vs 8 aujourd'hui) qui produit un score continu et une décision préliminaire **HEDGE** **MONITOR** **OPEN** . Lisible coefficient par coefficient, donc débogable à la main si une décision surprend.
2. **Stage B — la garde de confiance.** Un classifieur GBM (sklearn `GradientBoostingClassifier`) qui regarde le contexte du jour (volatilité du score, régime récent, jour de la semaine, accord macro/voi...) et estime une probabilité p^* que la décision de Stage A soit correcte. Si $p^* < 0.55$, on rétrograde en MONITOR ; sinon on garde la décision active. Le GBM ne produit jamais une décision — il filtre celle de Stage A.

Le nouvel algo s'appelle `accuracy v3.0.0` . Il est déployé en parallèle de v1.0.1 pendant 1 semaine en shadow mode — concrètement, le nightly écrit ses propres rows en DB sous la version v3.0.0, mais le dashboard et le compass-brief continuent à lire v1.0.1. On compare les deux séries chaque matin (cf. § 6.1). Si v3.0.0 fait mieux, on bascule par un simple flip de flag SQL (`pl_algorithm_version.is_active`) — pas de redéploiement de code, pas de rebuild d'image, pas de touch au frontend.

Côté utilisateur, rien ne change visuellement : même UI, même format de signal quotidien, même brief audio. Seule la qualité des recommandations augmente. La métrique de pilotage reste la même qu'aujourd'hui — **précision** (taux de signaux actifs dont la direction se confirme à J+1) — parce que c'est elle qui détermine la valeur ajoutée perçue par le client final.

GAIN ATTENDU

Précision walk-forward 6 mois : **~30% → 60%**, avec un taux de MONITOR contenu à **36%** (objectif utilisateur $\leq 60\%$).

Honnêteté intellectuelle. Sur avril 2026 spécifiquement (mois de rallye), on passe de 31% à 45% — nettement mieux mais loin de la cible 75%. Le plafond actuel ~60% est posé par le feature set existant. Casser ce plafond demandera de nouvelles features (OI delta, term structure, COT positioning) — c'est le ticket R&D suivant.

PRÉCISION AVRIL 26

31% → 45%

+13.8pp vs legacy

PRÉCISION WF 6M

~30% →

60%

$\tau = 0.55$

MONITOR RATE WF

36%

cible $\leq$ 60%

Pourquoi cette release ?

Le déclencheur

En avril 2026, le marché cocoa a fait un rallye de **+8%** (2470 → ~2670 USD/tonne sur le contrat front CAK26). Le système production a été coincé sur **HEDGE** pendant la majeure partie de la montée — c'est-à-dire qu'il recommandait de couvrir l'exposition long alors que le sous-jacent grimpait. Pour un client physique acheteur, cela revient à se couvrir au mauvais moment et payer le coût du hedge sans bénéficier du rallye. Concrètement :

ALGO EN DB	PRÉCISION AVRIL 2026	MONITOR	HITS ACTIFS
legacy v1.0.1 — production active	31.2%	23.8%	5/16
power10years v2.0.0 — en DB, inactive	28.6%	33.3%	4/14

Avril était hors du train set ET hors du validation set du précédent optimizer (v8.2 multirun , calibré sur 2023–2025). Le système n'avait jamais « vu » de rallye macro de cette amplitude pendant son fitting — d'où l'effondrement de précision dès qu'un régime non-vu est apparu. Le polynôme legacy a en outre deux coefficients dominants ($p = -5.8$ sur la macroéco, $1 = 5.9$ sur le ratio volume/open-interest) avec des exposants > 1.8 qui saturent le score dès que les inputs s'écartent un peu de leur moyenne. Résultat : un macro positif quel qu'il soit pousse mécaniquement le score vers HEDGE, indépendamment du contexte technique. C'est exactement le comportement anti-rallye observé.

Le diagnostic en trois points

- Polynôme mal calibré pour les régimes haussiers.** Le feature set est trop pauvre (8 indicateurs, dominés par 2) et les exposants saturent. Toute correction par re-fitting des coefficients sur la donnée existante reproduira le même biais — il faut plus de signal, pas mieux ajusté.
- Seuils asymétriques.** La zone OPEN couvre une plage de score 2× plus large que la zone HEDGE. À score absolu équivalent, on est plus enclin à dire OPEN qu'à dire HEDGE — ce qui amplifie l'erreur en régime baissier ; et symétriquement, en régime haussier, on tombe en HEDGE pour des magnitudes que la zone OPEN aurait absorbées.
- Le score n'est pas monotone avec la justesse.** Contre-intuitivement, le quintile de scores les plus élevés en valeur absolue n'est pas le plus précis : sur l'historique, c'est le quintile le plus faible qui l'emporte (60.7% vs 55.6%). Donc une garde simple de type « rejette les signaux faibles » est inadaptée — il faut une fonction non-linéaire du contexte (cf. Stage B).

Le travail R&D

Six mois de notebooks consolidés en une stratégie en deux étages. La trajectoire : on a d'abord essayé d'enrichir uniquement le polynôme (de 8 à 16 features), puis testé une logistique sur le score absolu (AUC 0.45 — bruit), puis le GBM (cliff naturel à $\tau=0.55$). Chaque étape — y compris les pistes abandonnées (gate sur `|score|`, ensemble averaging, isotonic regression sur le score) — est documentée dans `COMPASS_V9_PROD_INTEGRATION.md § Research journey`. Cette release est le premier livrable consolidant ces six mois de R&D en code production.

03 · ARCHITECTURE

Comment ça marche, pédagogiquement

L'idée centrale : un signal de marché est **composé** de deux questions différentes. Premièrement, « quel est le score brut basé sur les indicateurs techniques et macro du jour ? » — c'est une question d'agrégation, où l'on veut une fonction lisible et stable. Deuxièmement, « ce score est-il fiable aujourd'hui, vu le contexte récent ? » — c'est une question de méta-confiance, où la non-linéarité et la mémoire courte sont nécessaires. Le système actuel mélange les deux dans un seul polynôme et c'est en partie pour ça qu'il sature. `accuracy v3.0.0` sépare explicitement ces deux questions en deux étages, ce qui rend chaque couche plus simple à valider, à déboguer et à itérer.

3.1 — Stage A : le nouveau polynôme V9 (16 features)

Même formalisme que `legacy` :

```
score = k + Σ coef_i × sign(x_i) × |x_i|^exp_i
```

Pourquoi cette forme ? `x_i` est le z-score d'un indicateur (RSI, MACD, etc.) — c'est-à-dire l'écart à sa moyenne mobile 252 jours, divisé par son écart-type. La standardisation rend les features comparables dans le temps et entre instruments. `sign × |·|^exp` permet une non-linéarité douce : un exposant < 1 atténue les valeurs extrêmes (utile pour les indicateurs bruyants), > 1 les amplifie (utile pour des indicateurs de conviction). C'est le bon compromis interprétabilité / expressivité — chaque coefficient répond à une question précise (« quel poids RSI ? quelle non-linéarité sur RSI ? ») et tient sur une ligne de log.

Mais on passe de 8 features à 16 :

	FEATURES (8) — DÉJÀ EN PL_INDICATOR_DAILY	FEATURES (8) — CALCULÉES EN PYTHON
Z-scores rolling 252j	rsi, macd, stoch, atr, close/pivot, vol/oi	—
Binaires / signe	momentum, macroeco	rsi_extreme (rsi >1.5), voi_macro_agree (signes alignés)
Régime (mémoire courte)	—	regime_20d (signe close – close[-20]), regime_5d
Saturation (anti- domination)	—	voi_squashed (tanh(voi)), macro_squashed (tanh(macro))
Interactions	—	rsi × voi, macd × sign(momentum)

Pourquoi 16 features au lieu de 8 ? Le legacy était dominé par 2 termes (macro et voi) avec des exposants > 1.8 — ils saturent dès $|z| \geq 1$, ce qui fait qu'à partir d'un macro un peu fort, plus rien d'autre ne peut faire bouger la décision. Les nouvelles features font deux choses : (i) elles ajoutent du **contexte** qui n'existait pas avant — régime des 5/20 derniers jours, accord ou désaccord entre macro et flux (voi_macro_agree), RSI extrême, interactions ; et (ii) elles **bornent** les contributions dominantes via tanh, qui plafonne naturellement à ± 1 , empêchant un seul indicateur de prendre toute la place. Plus de signal disponible pour la même classe de modèle, sans changer de paradigme.

Pourquoi pas remplacer le polynôme par un GBM end-to-end ? Interprétabilité et auditabilité. Chaque coefficient mappe à un indicateur connu de l'équipe, ça tient en 35 floats dans pl_algorithm_config, et un signal aberrant peut se déboguer à la main en 5 minutes : on regarde quel terme contribue le plus, on rapproche du contexte du jour, on comprend. Avec un GBM end-to-end, le débogage devient un travail de SHAP plots et d'intuition. Le ML reste donc cantonné à la couche confiance (Stage B) où la non-linéarité est essentielle et où l'on peut se permettre de moins comprendre pourquoi tant qu'on contrôle quand on lui fait confiance (cf. § 6.3, monitoring du $\hat{p}$).

3.2 — Stage B : la garde GBM (15 features)

Une fois que Stage A a produit son score V9 et sa décision préliminaire (**HEDGE** / **MONITOR** / **OPEN**), on ne lui fait pas confiance aveuglément. La règle de décision finale est asymétrique :

- **Si MONITOR** : on garde MONITOR (rien à filtrer — on ne « réveille » jamais un signal passif).
- **Si HEDGE ou OPEN** : on calcule 15 features de contexte du jour, on les passe à un GradientBoostingClassifier entraîné sur l'historique walk-forward. Le modèle sort une probabilité $\hat{p} \in [0, 1]$ que la décision V9 soit correcte (i.e. que le close à J+1 confirme la direction). Si $\hat{p} \geq 0.55$ on garde la décision active ; sinon, **on rétrograde en MONITOR**.

Concrètement, le GBM répond à la question : « sachant que Stage A dit OPEN avec un score de 0.8, et sachant qu'on est lundi, que la volatilité du score sur 20j est élevée, que les 3 derniers signaux ont raté et que le régime 20j est négatif... est-ce qu'on devrait quand même publier OPEN ? ». Si le GBM

dit $\hat{p} = 0.42$, on rétrograde en MONITOR — le contexte est trop hostile pour faire confiance au signal brut. Si $\hat{p} = 0.71$, on publie OPEN.

CATÉGORIE	FEATURES GBM (15)
Score V9	score (raw), abs_score_norm ($ score /std$), margin_to_active (distance au seuil le plus proche)
Contexte du jour	voi_norm, regime_20d, regime_5d, rsi_x_voi, voi_macro_agree
Mémoire courte	score_vol_20d (volatilité du score sur 20j), score_minus_ema5 (déviations à la moyenne 5j), prior_3d_hit (taux de réussite sur les 3 derniers signaux)
Saisonnalité hebdo	dow_mon, dow_tue, dow_wed, dow_thu (vendredi = baseline)

Pourquoi un GBM et pas une régression logistique ? Une logistique a été testée — AUC fold-by-fold de 0.45–0.47, soit essentiellement du bruit. La raison est mécanique : le GBM exploite la **non-monotonicité** du score V9 mentionnée § 2 (le quintile le plus précis est le plus bas, 60.7% vs 55.6%). Une fonction linéaire ne peut pas capturer un signal en U ; un GBM, par construction par arbres, oui. Plus généralement, le GBM est capable d'apprendre des règles conditionnelles du type « si le régime 20j est négatif ET qu'on est en début de semaine ET que la volatilité du score est haute, alors méfie-toi des signaux faibles » — ce sont exactement les patterns que les notebooks ont fait émerger.

Pourquoi $\tau = 0.55$ et pas 0.50 ou 0.60 ? C'est l'inflexion de la courbe Pareto précision/MONITOR observée sur le walk-forward. Sous 0.55, la précision plafonne à 54% (le gate ne filtre pas assez). À 0.55, on est à 60.0% précision avec 36% MONITOR — le sweet spot. Au-dessus de 0.55, on chute brutalement à 96.5% de MONITOR (le modèle ne valide quasiment plus rien — utile en théorie, inutilisable en production où l'utilisateur attend des signaux actifs régulièrement). Le τ choisi est un cliff naturel du modèle, pas un paramètre arbitraire — c'est important parce que ça veut dire qu'il restera relativement stable au prochain retrain, tant qu'on garde la même distribution de features.

3.3 — Comment Stage A et Stage B s'imbriquent dans le nightly

Le pipeline nightly fait 3 passes (au lieu de 2 aujourd'hui). Chaque passe a une responsabilité unique et écrit des colonnes spécifiques en DB — pas de double-écriture, pas de course de données :

```

19:15 cc-compute-indicators   écrit les scores Stage A (legacy + accuracy)
19:20 cc-daily-analysis       écrit macroeco_bonus (LLM macro)
19:25 cc-accuracy-finalize    - NOUVEAU : Stage B GBM gate
                              ré-applique le polynôme V9 avec macro fraîche,
                              puis applique la garde GBM,
                              écrit decision + confidence sur les rows v3.0.0.
19:30 cc-compass-brief       lit la version active (audio NotebookLM)
```

Pourquoi un job dédié à 19:25 et pas dans `compute-indicators` ? Parce que le Stage A utilise `macroeco_bonus` comme une de ses 16 features, et cette colonne est écrite par `cc-daily-analysis` à 19:20. À 19:15, `compute-indicators` n'a que la macro de la veille — il produirait donc un score V9 « rancis » d'un jour. La solution propre : on laisse `compute-indicators` produire un Stage A préliminaire (utile pour les rows legacy, et de toute façon réécrit pour les rows v3.0.0), puis `cc-accuracy-finalize` fait la passe finale à 19:25 avec macro fraîche + garde GBM. Cela respecte la rule projet `pipeline-continuity.md` : chaque writer reçoit ce dont il a besoin, n'invente rien.

Pourquoi nightly et pas intraday ? Parce que le scope produit est « décision quotidienne après close européen », pas trading haute fréquence. Le close du contrat front (Londres, 18h45 CET) est la donnée principale. À 19:15 on est sûr d'avoir le close consolidé, à 19:25 on a la macro LLM, à 19:30 le brief audio est prêt pour 06:00 le lendemain matin chez l'utilisateur. Toute la chaîne tient en 15 minutes une fois par jour ouvré — c'est cohérent avec le besoin métier et minimise le coût Cloud Run (cf. § 8.B).

Ce qu'on déploie concrètement

4.1 — Schéma DB (1 migration Alembic, idempotente)

ÉLÉMENT	QUOI
<code>pl_algorithm_version</code>	<code>INSERT name='accuracy', version='3.0.0', is_active=false, compute_enabled=true</code>
<code>pl_algorithm_config</code>	<code>INSERT 37 rows: k + 16 × (<feature>_coef, <feature>_exp) + hedge_threshold + open_threshold</code>
<code>pl_confidence_model</code> (NEW table)	Stocke l'URI GCS du joblib + SHA-256 + <code>score_scale</code> + <code>threshold</code> + métadonnées de training
Index unique (NEW)	<code>WHERE is_active = true</code> sur <code>pl_algorithm_version</code> — ferme le risque actuel de 2 versions actives en même temps

Rien à modifier dans `pl_indicator_daily` : la colonne `confidence DECIMAL(5,2)` existe déjà (utilisée aujourd'hui pour le score LLM, on l'écrasera avec `p^× 100` pour les rows v3.0.0).

4.2 — Code (nouveaux fichiers)

```

backend/
  alembic/versions/<hash>_add_accuracy_v3_and_confidence_model.py ← migration
  app/engine/accuracy/                                           ← stage A (16-
feature)
    config.py           # AccuracyConfig dataclass + from_db_rows()
    features.py         # 8 derived features (vectorized)
    polynomial.py       # compute_score, compute_decision (16-feature path)
    pipeline.py         # AccuracyPipeline (orchestrator)
  app/models/confidence_model.py                                ← PlConfidenceModel
  scripts/accuracy_train/                                       ← training (one-off +
monthly)
    main.py + data_loader / feature_builder / trainer / artifact_writer
  scripts/accuracy_finalize/                                    ← stage B (nightly
job)
    main.py + gbm_loader / feature_pipeline / db_writer / cold_start
  tests/engine/test_accuracy_features.py
  tests/engine/test_accuracy_polynomial.py
  tests/scripts/test_accuracy_finalize.py

```

Le code legacy n'est pas touché. L'engine `runner.py` aiguille vers la bonne stratégie selon `algorithm_name` (`"legacy"` → ancien chemin, `"accuracy"` → nouveau).

4.3 — Infra (1 nouveau job + 1 nouveau cron + 3 deps Python)

- `backend/pyproject.toml` : ajouter `scikit-learn ^1.6` , `joblib ^1.4` , `google-cloud-storage ^2.18` . (`pandas` , `numpy` déjà présents.)
- `.github/workflows/deploy.yml` : ajouter `cc-accuracy-finalize` à la boucle des jobs Cloud Run (passe de 8 → 9 jobs). Réutilise `Dockerfile.jobs` .
- `infra/terraform/scheduler.tf` : nouveau cron `25 19 * * 1-5` qui trigger `cc-accuracy-finalize` .
- GCS bucket `cacao000-models` : créé via Terraform si absent, IAM `roles/storage.objectViewer` sur `cc-cloud-run-jobs` SA.

05 · MISE EN PRODUCTION

Rollout en 3 phases

PHASE	DURÉE	ACTION	CRITÈRE PASS
1. Shadow	5 jours ouverts	v3.0.0 produit ses décisions, v1.0.1 reste active. Dashboard et compass-brief inchangés.	≥ 1 hit de plus que v1.0.1, OU même nb de hits avec $\leq 80\%$ du MONITOR de v1.0.1
2. Cutover	Day 6	<code>UPDATE pl_algorithm_version</code> en transaction : <code>flip is_active</code> .	Dashboard et compass-brief lisent automatiquement v3.0.0 dès le query suivant
3. Steady-state	Continu	Monitoring sur 3 axes (cf. § 6)	Aucun seuil d'alerte déclenché pendant 30 jours

NOTE on saute le vrai A/B (week 2 du plan original) — il n'y a qu'un contrat actif à la fois et pas de feature-flag frontend. Un A/B « subset of contracts » ne ferait aller que CAK26 vs lui-même. Shadow → cutover direct, plus pragmatique.

Rollback

Une seule transaction SQL (1 ligne par version) :

```
BEGIN;
  UPDATE pl_algorithm_version SET is_active = true WHERE name='legacy' AND
version='1.0.1';
  UPDATE pl_algorithm_version SET is_active = false WHERE name='accuracy' AND
version='3.0.0';
COMMIT;
```

Le job `cc-accuracy-finalize` continue à tourner et à écrire sur les rows v3.0.0 (pour comparaison post-mortem), mais le dashboard et compass-brief reviennent immédiatement sur v1.0.1. Pour stopper le job : `gcloud scheduler jobs pause cc-accuracy-finalize-trigger` .

Métriques de succès chiffrées

6.1 — Critères de pass shadow → cutover (5 jours)

Le résultat de la comparaison shadow se mesure sur 3 nombres simples — extraits via la query SQL ci-dessous lancée chaque matin :

```
SELECT
  i1.date,
  i1.decision      AS legacy_decision,
  i3.decision      AS accuracy_decision,
  i3.confidence    AS accuracy_p_hat,
  d.close,
  LEAD(d.close) OVER (PARTITION BY i1.contract_id ORDER BY i1.date) AS close_t1
FROM pl_indicator_daily i1
JOIN pl_indicator_daily i3 USING (date, contract_id)
JOIN pl_algorithm_version v1 ON v1.id = i1.algorithm_version_id AND v1.name='legacy'
JOIN pl_algorithm_version v3 ON v3.id = i3.algorithm_version_id AND v3.name='accuracy'
JOIN pl_contract_data_daily d USING (date, contract_id)
WHERE i1.date >= CURRENT_DATE - 7;
```

PASS CRITERION

MÉTRIQUE	SEUIL PASS	ACTION SI FAIL
Hits accuracy – Hits legacy	≥ +1 sur 5 jours	Investigate, ne pas cutover
MONITOR rate accuracy / MONITOR rate legacy	≤ 0.80 (si hits égaux)	Investigate
p [^] distribution	médiane ∈ [0.40, 0.65]	Si dérive, retrain immédiat avant cutover

Si pass critères atteints → cutover. **Sinon** → debug (le plus probable : feature stage B ou macroeco mal-écrite par daily-analysis).

6.2 — Cibles attendues (R&D walk-forward Nov 2025 → Avr 2026)

MOIS	V9 ACTIVE	KEPT ACTIVE (GATE T=0.55)	HITS	PRÉCISION SUR KEPT
2025-11	13	13	8	61.5%
2025-12	10	10	7	70.0%
2026-01	15	8	3	37.5%
2026-02	9	8	8	100%
2026-03	18	18	7	38.9%
2026-04	21	20	9	45.0%
WF concat	86	77	42	54.5% @ $\tau=0.50$
WF concat	—	55	33	60.0% @ $\tau=0.55$

6.3 — Alertes monitoring continu (post-cutover)

3 panels GCP Monitoring (logs structurés émis par `cc-accuracy-finalize`) :

MÉTRIQUE	PÉRIODE	SEUIL ALERTE
Decision distribution	rolling 5 jours	MONITOR > 70% (gate trop agressif) ou < 15% (gate trop lax)
Précision active	rolling 30 jours	< 50% (cible 60%)
$p^{\wedge}$ médiane	rolling 30 jours	dérive > ± 0.05 vs distribution training (signal de concept drift → retrain)

Cadence retrain : **mensuelle** par défaut (cron manuel `poetry run accuracy-train`), **immédiate** si l'alerte 3 se déclenche. Chaque retrain = nouvelle row dans `pl_confidence_model` avec `is_active=true` (ancienne row reste pour audit).

6.4 — Tests d'acceptance pré-deploy (CI)

À ajouter en `pytest -m accuracy_acceptance` — bloquent le merge :

#	TEST	EXPECTED
1	Polynomial reproduction sur avril 2026	9 hits / 21 active (égalise le notebook)
2	GBM reproduction (random_state=42, $\tau=0.55$)	~9 hits / 20 kept / 1 dropped
3	Cold-start safety (contrat avec 0 history)	Décision préliminaire conservée (pas de MONITOR par défaut)
4	Schema integrity	pl_algorithm_config a 37 rows pour v3.0.0
5	Rollback	Transaction → cc-daily-analysis produit identique à v1.0.1
6	Active-version uniqueness	Tentative de 2 versions actives → rejet par index unique

07 · LIMITATIONS

Limitations honnêtes

À présenter avant approbation.

LIMITATION 1 — APRIL 2026 RESTE À 45 %

Le +15pp WF gain est concentré sur les mois calmes (févr 100%). Si avril prochain reproduit le rallye, on aura toujours une précision moyenne. **La garde GBM n'est pas suffisante pour les régimes shifts.**

LIMITATION 2 — AUC FOLD-BY-FOLD DU GBM ≈ 0.45

Légèrement anti-prédictif. Le modèle exploite une non-monotonie, pas un signal fort. Robustesse cross-régime incertaine.

LIMITATION 3 — PLAFOND D'INFORMATION $\sim 60\%$

Sur le feature set actuel. Casser ce plafond = ticket R&D ouvert : enrichir Stage B avec OI delta, term structure (basis front vs second-month), COT positioning (`com_net_us` est déjà en DB mais inexploité). Estimation +10–15pp précision, 1–2 jours de pipeline.

LIMITATION 4 — YTD PERFORMANCE HARDCODÉE À 83.47 %

Dans le dashboard (`dashboard_service.py:256-259`) suite au commit `4c83569` . Le calcul réel reste codé mais inatteignable. Restauration = ticket séparé, **pas bloquant pour v3.0.0.**

Annexes

A. Comparatif chiffré recap

APPROCHE	PRÉCISION AVRIL 2026	PRÉCISION WF CONCAT	MONITOR AVRIL	MONITOR WF
legacy v1.0.1 (production actuelle)	31.2%	~30%	23.8%	—
power10years v2.0.0 (R&D, jamais shippé)	28.6%	—	33.3%	—
V9 polynôme seul (16 features, sans gate)	42.9%	~50%	0%	28%
Gate sur <code> score </code> à $\sigma=2$ (baseline naïve)	75.0%	—	81.0% (off-cap)	—
accuracy v3.0.0 (V9 + GBM gate $\tau=0.55$)	45.0%	60.0%	5%	36%

B. Coût opérationnel (ordre de grandeur)

- 1 nouveau Cloud Run Job nightly (~5 min, 1 GiB RAM) : négligeable (< 1 €/mois Cloud Run).
- 1 GCS bucket pour les artefacts joblib (~50 KB × ~12 retrain/an) : négligeable.
- 0 nouveau service / API endpoint / DB instance.

C. Liens

- Spec complète : COMPASS_V9_PROD_INTEGRATION.md
- Plan d'implémentation détaillé : `~/ .claude/plans/tu-peux-etudier-le-resilient-fairy.md` (local)
- Rules projet pertinentes :
 - `pipeline-error-handling.md` — fail loud, no silent retry
 - `pipeline-continuity.md` — writers reçoivent, n'inventent pas
 - `north-star-alignment.md` — config-as-data, contract-centric



COMMODITIES COMPASS · ACCURACY V3.0.0 · MAI 2026 · CONFIDENTIEL